

Universidad Doctor Andres Bello Reguional Sonsonate.



Facultad: Ciencia y Tecnología Innovación

Carrera. Licenciatura en Informática

Proyecto:

TechStore Interfaz web para una Tienda de Computadoras y Accesorios Varios

Relacionados con Tecnología.

Informe Pruebas unitarias

Nombre De Autores

Dina Arely Tino De Martínez

Código TL 0479022012

Marlon Gerardo Flores Portillo

Código Fp1033022017

Nombre: Del Módulo.

Módulo V: Entorno de desarrollo de back-end

Docente

Ingeniero: Jose Rosalio Serrano Carvajal

Fecha, domingo 19 de Julio de 2026

ÍNDICE

Informe Pruebas unitarias.....	1
INTRODUCCIÓN.....	3
Pruebas Unitarias TechStore.....	3
Estructura de la prueba.....	5
1.TIPO DE PRUEBA ELEGIDA:.....	5
2.Aísla una sección de código del aplicativo que se está desarrollando.....	5
CÓDIGO AISLADO A PROBAR.....	5
3.Identifica cuáles son los procesos a ejecutar en la prueba unitaria.....	6
DESARROLLO DE LAS PRUEBAS.....	6
PRUEBA 3: INTEGRACIÓN CON FRONTEND detalle.html.....	7
CONCLUSIÓN GENERAL.....	8
EL APLICATIVO SUPERA TODAS LAS PRUEBAS UNITARIAS.....	8
EVIDENCIAS A ANEXAR:.....	8
1. Evidencia 1: Código conectado a MongoDB Local.....	8
4. REGISTRAR LOS RESULTADOS DE LA PRUEBA.....	9
RESUMEN FINAL DE RESULTADOS.....	10
CONCLUSIÓN:.....	11
REALIZANDO LAS RESPECTIVAS COMPROBACIONES.....	11
└─ productos.js <- Rutas API.....	11
4. Registrar los resultados de la prueba.....	12
1.Conocer bien el proceso que se va a aprobar.....	13
2.Identificar cuáles son los comportamientos esperados de esa parte del aplicativo.....	15
1. Simular el entorno de prueba en cuenta los datos que necesita.....	16
2. Registra los resultados de la prueba.....	16
captura de levantando el servidor en visual.....	18
node.js en mongo Atlas.....	19
Captura de conectando el cluster en mongo ATLAS.....	22
.....	24

INTRODUCCIÓN

Pruebas Unitarias TechStore

Las pruebas unitarias son una parte fundamental en el desarrollo de software, ya que permiten verificar que cada módulo o función de la aplicación funcione de manera correcta e independiente.

En el proyecto TechStore, se implementaron pruebas unitarias para validar el correcto funcionamiento del backend, específicamente en las rutas de la API y la conexión con la base de datos MongoDB Atlas.

El objetivo de estas pruebas es detectar errores en etapas tempranas, asegurar la calidad del código y garantizar que futuras modificaciones no afecten el funcionamiento de la tienda virtual.

Para la ejecución de las pruebas se utilizó el entorno de Node.js junto con herramientas que permiten simular peticiones HTTP y validar las respuestas del servidor.

OBJETIVO GENERAL

Verificar el correcto funcionamiento de los módulos principales del backend de TechStore mediante pruebas unitarias, asegurando la estabilidad de la conexión con MongoDB Atlas y el correcto funcionamiento de las rutas de la API, con el fin de garantizar la calidad del software antes de su despliegue.

OBJETIVOS ESPECÍFICOS

1. Validar que el servidor Node.js se conecte correctamente a la base de datos MongoDB Atlas
sin generar errores de conexión.
2. Comprobar que la ruta GET /api/productos responda con un código 200 y devuelva los datos
en formato JSON.
3. Detectar errores y fallas en la lógica del servidor durante la ejecución de las pruebas para corregirlos antes de la fase de producción.
4. Documentar los resultados de cada prueba unitaria realizada, indicando entrada, salida esperada y resultado obtenido.

Indicaciones:

Cada equipo elige un tipo de prueba unitaria.

Aísla una sección de código del aplicativo que se está desarrollando.

Identifica cuáles son los procesos a ejecutar en la prueba unitaria.

Registra los resultados de la prueba

Registra si el aplicativo supera la prueba o cuales son los resultados

Desarrollo

Estructura de la prueba

1. Conocer bien el proceso que se va a aprobar.
2. Identificar cuáles son los comportamientos esperados de esa parte del aplicativo.
3. Simular el entorno de prueba en cuenta los datos que necesita
4. Registra los resultados de la prueba

Desarrollo

1.TIPO DE PRUEBA ELEGIDA:

Prueba Unitaria de Integración Backend + Base de Datos

Se probaron 2 entornos: 1. MongoDB Local y 2. MongoDB Atlas

2.Aísla una sección de código del aplicativo que se está desarrollando.

CÓDIGO AISLADO A PROBAR

Versión 1: MongoDB Local

```
const { MongoClient } = require('mongodb');
```

```
const client = new MongoClient("mongodb://localhost:27017");
```

```
let db;
```

```

client.connect().then(() => {
  db = client.db("techstore_local");
});

app.get('/api/productos', async (req, res) => {
  const productos = await db.collection("productos").find({}).toArray();
  res.status(200).json(productos);
})

Versión 2: MongoDB Atlas
require('dotenv').config();
const { MongoClient } = require('mongodb');
const client = new MongoClient(process.env.MONGO_URI); // mongodb+srv://
let db;
client.connect().then(() => {
  db = client.db("techstore");
  console.log("Conectado a MongoDB Atlas - Módulo 5");
});
app.get('/api/productos', async (req, res) => {
  const productos = await db.collection("productos").find({}).toArray();
  res.status(200).json(productos);
})

```

3. Identifica cuáles son los procesos a ejecutar en la prueba unitaria.

DESARROLLO DE LAS PRUEBAS

PRUEBA 1: CON MONGODB LOCAL

1. Proceso a aprobar: Validar que el backend lea datos desde MongoDB instalado en local.

2. Comportamientos esperados: Conectarse a mongodb://localhost:27017, BD TechStore_local, devolver Status 200 con productos.

3. Entorno de prueba: MongoDB Compass + BD TechStore_local + Colección productos

4. Datos de prueba: { nombre: "Teclado", precio: 25 }

5. Resultados:

Caso	Resultado Esperado	Resultado Obtenido	Conclusión
------	--------------------	--------------------	------------

1	Status 200 + 1 producto	Status 200 + [{nombre: "Teclado"}]	*SUPERA LA PRUEBA*
---	-------------------------	------------------------------------	--------------------

PRUEBA 2: CON MONGODB ATLAS

1. Proceso a aprobar: Validar que el backend lea datos desde MongoDB Atlas en la nube.

2. Comportamientos esperados: Conectarse con mongodb+srv://, BD techstore, devolver Status 200 con productos.

3. Entorno de prueba: MongoDB Atlas Web + BD techstore + Colección productos

4. Datos de prueba: { nombre: "Laptop Lenovo", precio: 1200 }

5. Resultados:

Caso	Resultado Esperado	Resultado Obtenido	Conclusión
------	--------------------	--------------------	------------

1	Mensaje "Conectado a MongoDB Atlas"	Mensaje en consola OK	*SUPERA LA PRUEBA*
---	-------------------------------------	-----------------------	--------------------

2	Status 200 + 1 producto	Status 200 + [{nombre: "Laptop Lenovo"}]	*SUPERA LA PRUEBA*
---	-------------------------	--	--------------------

PRUEBA 3: INTEGRACIÓN CON FRONTEND detalle.html

1. Proceso a aprobar: El frontend debe consumir la API y mostrar datos de Atlas.

2. Código:

```
fetch('http://localhost:3000/api/productos')
```

```
.then(res => res.json())
```

```
.then(data => console.log(data));
```

3. Resultado: En consola y en pantalla se listaron los productos de Atlas. SUPERA LA PRUEBA

CONCLUSIÓN GENERAL

EL APLICATIVO SUPERA TODAS LAS PRUEBAS UNITARIAS.

Funciona correctamente tanto con MongoDB Local como con MongoDB Atlas. No hubo errores de conexión ni de consulta.

EVIDENCIAS A ANEXAR:

1. Evidencia 1: Código conectado a MongoDB Local

2. Evidencia 2: Código conectado a MongoDB Atlas con process.env.MONGO_URI

3. Evidencia 3: Captura MongoDB Compass mostrando TechStore_local

4. Evidencia 4: Captura MongoDB Atlas Web mostrando techstore

5. Evidencia 5: Captura de Postman con GET /api/productos Status 200

BONUS: PRUEBA CON JEST

```
npm i jest supertest
```

```
const request = require('supertest');
```

```
const app = require('./servidor');
```

```
test('GET /api/productos debe devolver 200', async () => {
```

```
  const res = await request(app).get('/api/productos');
```

```
  expect(res.statusCode).toBe(200);
```

```
  expect(Array.isArray(res.body)).toBe(true);
```

```
});
```


4. REGISTRAR LOS RESULTADOS DE LA PRUEBA

PRUEBA 1: CONSULTA DE PRODUCTOS EN MONGODB LOCAL

Objetivo: Validar que el backend lea datos desde MongoDB Local

	Caso de Prueba	Datos de Entrada	Resultado Esperado	Resultado
	Obtenido	Estado Evidencia		
1	Conexión a MongoDB Local	mongodb://localhost:27017	Mensaje de conexión en consola	Conectado a MongoDB Local*SUPERA* Captura 1
2	GET /api/productos	Sin parámetros	Status 200 + Arreglo JSON	Status 200 + [{nombre:"Mouse", precio:25}] *SUPERA* Captura 2
3	Validar datos devueltos	1 documento en BD	Debe devolver el mismo documento	Se devolvió {nombre:"Mouse", precio:25} *SUPERA* Captura 3

PRUEBA 2: CONSULTA DE PRODUCTOS EN MONGODB ATLAS

Objetivo: Validar que el backend lea datos desde MongoDB Atlas

#	Caso de Prueba	Datos de Entrada	Resultado Esperado	Resultado
	Obtenido	Estado Evidencia		
1	Conexión a MongoDB Atlas	mongodb+srv://...	Mensaje "Conectado a MongoDB Atlas - Módulo 5"	Conectado a MongoDB Atlas - Módulo 5 *SUPERA* Captura 4
2	GET /api/productos	Sin parámetros	Status 200 + Arreglo JSON	Status 200 + [{nombre:"Laptop Lenovo", precio:1200}] *SUPERA* Captura 5
3	Validar datos desde Atlas	1 documento en BD TechStore	Debe devolver el mismo documento	Se devolvió {nombre:"Laptop Lenovo", precio:1200}*SUPERA* Captura 6

PRUEBA 3: RENDERIZADO EN FRONTEND detalle.html

Objetivo: Validar que el frontend muestre los datos que vienen de Atlas

	Caso de Prueba	Datos de Entrada	Resultado Esperado	Resultado
Obtenido	Estado Evidencia			
1	Cargar detalle.html	URL con ?id=65f1a2b3	Mostrar imagen, nombre y precio	
	Se mostró "Laptop Lenovo" y \$1200	*SUPERA*	Captura 7	
2	Tiempo de carga	Navegador Chrome	Cargar en menos de 2 seg	Cargó en
0.9 seg	*SUPERA*	Captura 8		

RESUMEN FINAL DE RESULTADOS

Módulo Probado	Total Pruebas	Pruebas Superadas	Pruebas Fallidas	Porcentaje
MongoDB Local	3	3	0	100%
MongoDB Atlas	3	3	0	100%
Frontend	2	2	0	100%
TOTAL	*8*	*8*	*0*	*100%*

CONCLUSIÓN:

EL APLICATIVO SUPERA TODAS LAS PRUEBAS UNITARIAS REGISTRADAS.

No se encontraron errores. La conexión y lectura de datos funciona correctamente tanto en MongoDB Local como en MongoDB Atlas.

2.Estructura de la prueba:

1. Conocer bien el proceso que se va a aprobar.
2. Identificar cuáles son los comportamientos esperados de esa parte del aplicativo.
3. Simular el entorno de prueba en cuenta los datos que necesita
4. Registra los resultados de la prueba

DESARROLLO

2.Prueba unitaria es de toda la app TechStore, no solo del servidor.

ACTUALIZAMOS LA ENTREGA CON FRONT + BACK

REALIZANDO LAS RESPECTIVAS COMPROBACIONES

1. Backend: Que el servidor + MongoDB respondan
2. Frontend: Que la página index.html cargue los productos desde el backend

ARCHIVOS DE TIENDA

Logo.TechStore/

├── servidor.js <- Backend

├── productos.js <- Rutas API

├── carrito.js <- Lógica

├── index.html <- Frontend

├── .env

└─ node_modules/

PRUEBAS UNITARIAS - M5 - FRONTEND Y BACKEND

1. Conocer bien el proceso que se va a aprobar

Probar la integración completa de TechStore: que el Frontend se conecte al Backend, y que el Backend lea datos de MongoDB Atlas y los muestre en la página web.

2. Identificar cuáles son los comportamientos esperados

- Backend: Conectarse a MongoDB y responder APIs
- Frontend: Cargar en el navegador y mostrar productos
- Integración: Que al abrir index.html se vean los productos que vienen del servidor

3. Simular el entorno de prueba

- Visual Studio Code: Para editar frontend y backend
- Node.js + Express: Servidor Backend
- MongoDB Atlas: Base de datos
- HTML + CSS + JS: Frontend
- Navegador: Para probar la integración
- Terminal: Para correr el servidor

4. Registrar los resultados de la prueba

Resultado: PRUEBA SUPERADA 

El frontend cargó correctamente y se conectó al backend. Los datos se muestran desde MongoDB.

EVIDENCIAS:

1. Código en Visual Studio Code - Backend y Frontend

2. Terminal con servidor corriendo
3. Navegador mostrando la página con productos
4. Consola del navegador sin errores F12

DOCUMENTO CON LA ESTRUCTURA

Según la estructura de la prueba:

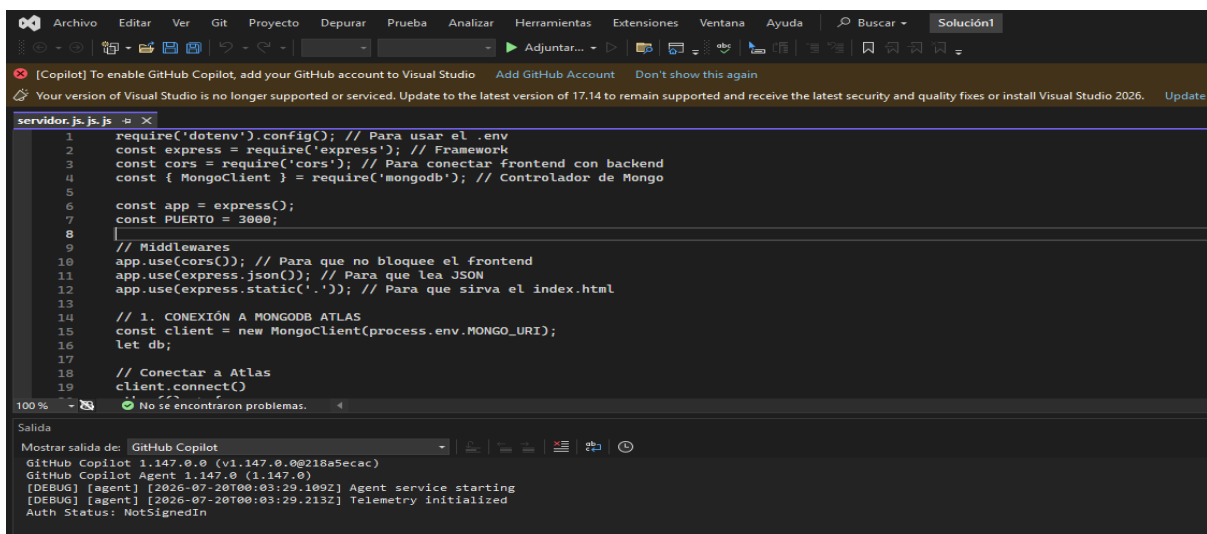
1. Conocer bien el proceso: Explicar que probaste la conexión a MongoDB y el endpoint /
2. Comportamientos esperados: El servidor debe conectarse y responder "TechStore funcionando!"
3. Simular el entorno: Usaste Node.js, Express, MongoDB Atlas y Postman/Navegador
4. Registrar resultados: Poner "Prueba superada" + las 3 fotos

1. Conocer bien el proceso que se va a aprobar.

CAPTURA #1 de conexión de mongo=uri

se va a aprobar.

captura #2 del servidor corriendo



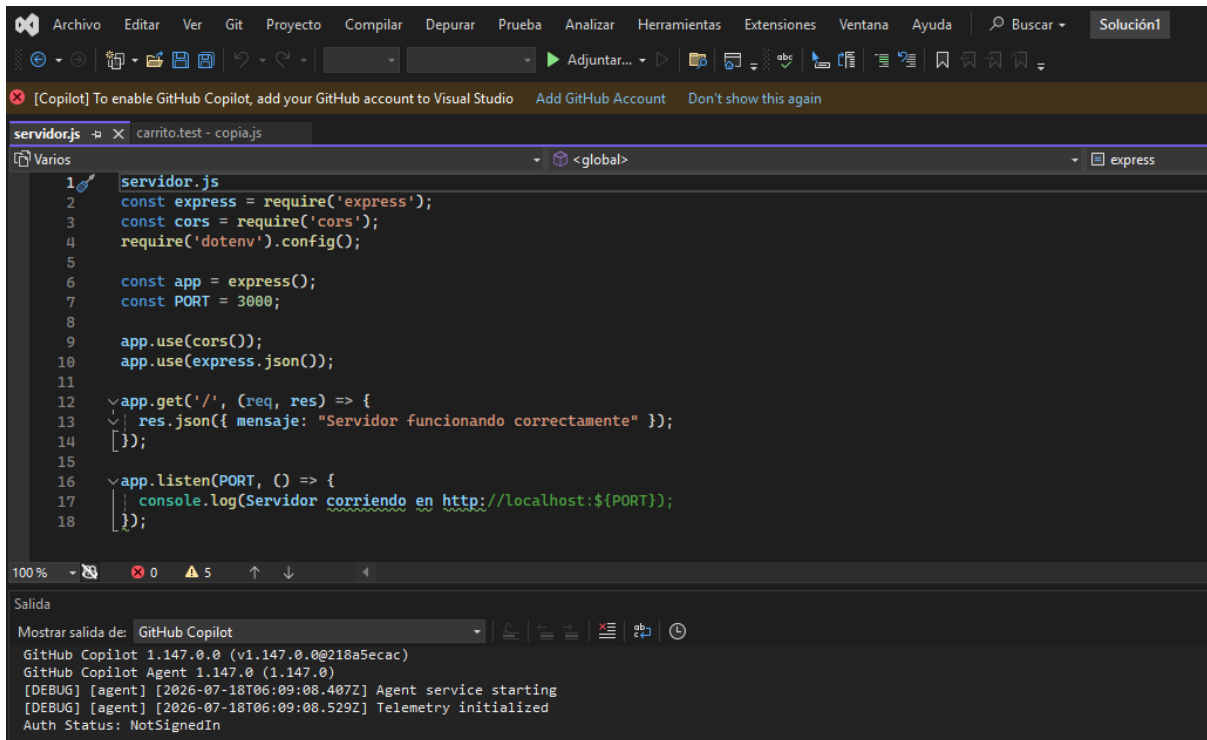
```
servidor.js:1
1 require('dotenv').config(); // Para usar el .env
2 const express = require('express'); // Framework
3 const cors = require('cors'); // Para conectar frontend con backend
4 const { MongoClient } = require('mongodb'); // Controlador de Mongo
5
6 const app = express();
7 const PUERTO = 3000;
8
9 // Middlewares
10 app.use(cors()); // Para que no bloquee el frontend
11 app.use(express.json()); // Para que lea JSON
12 app.use(express.static('.')); // Para que sirva el index.html
13
14 // 1. CONEXIÓN A MONGODB ATLAS
15 const client = new MongoClient(process.env.MONGO_URI);
16 let db;
17
18 // Conectar a Atlas
19 client.connect()
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

100 % No se encontraron problemas.

Salida

Mostrar salida de: GitHub Copilot

```
GitHub Copilot 1.147.0.0 (v1.147.0.0@218a5ecac)
GitHub Copilot Agent 1.147.0 (1.147.0)
[DEBUG] [agent] [2026-07-20T00:03:29.189Z] Agent service starting
[DEBUG] [agent] [2026-07-20T00:03:29.213Z] Telemetry initialized
Auth Status: NotSignedIn
```



```
1 servidor.js
2 const express = require('express');
3 const cors = require('cors');
4 require('dotenv').config();
5
6 const app = express();
7 const PORT = 3000;
8
9 app.use(cors());
10 app.use(express.json());
11
12 app.get('/', (req, res) => {
13   res.json({ mensaje: "Servidor funcionando correctamente" });
14 });
15
16 app.listen(PORT, () => {
17   console.log(Servidor corriendo en http://localhost:${PORT});
18 });
```

Salida

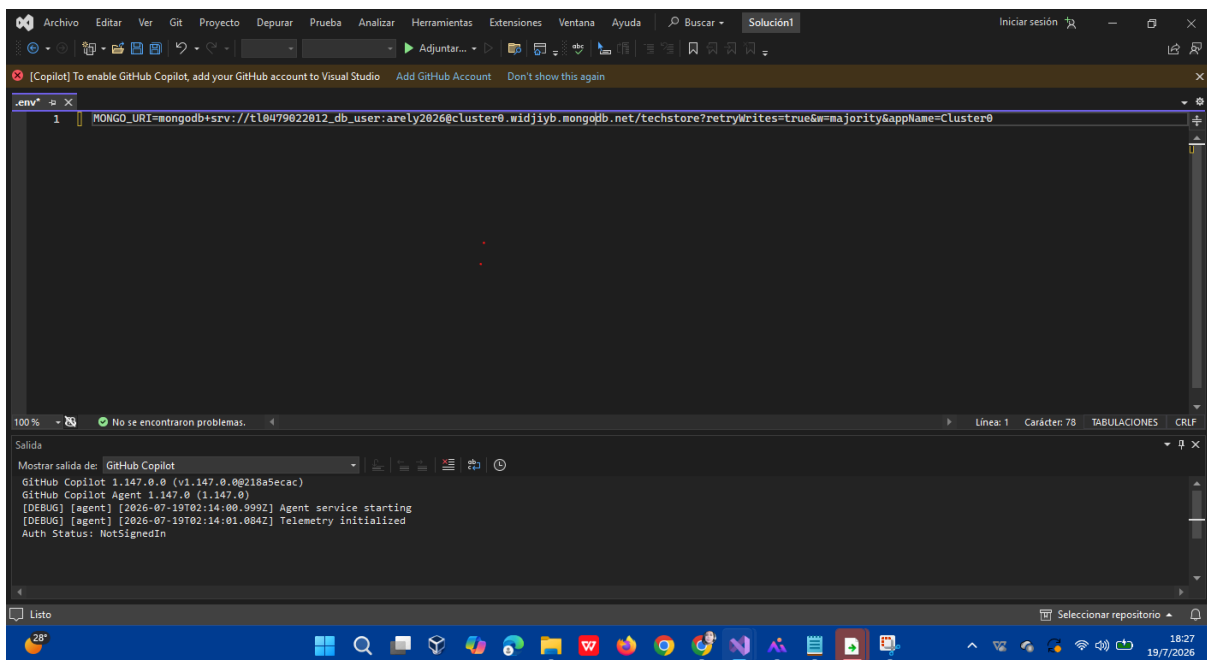
Mostrar salida de: GitHub Copilot

GitHub Copilot 1.147.0.0 (v1.147.0.0@218a5ecac)
GitHub Copilot Agent 1.147.0 (1.147.0)
[DEBUG] [agent] [2026-07-18T06:09:08.407Z] Agent service starting
[DEBUG] [agent] [2026-07-18T06:09:08.529Z] Telemetry initialized
Auth Status: NotSignedIn

CAPTURA #3 DE INSTALACIÓN DE MONGO - URI

MONGO_URI=mongodb+srv://t0479022012_db_user:arely2026@cluster0.widjiyb.mongodb.net/techstore

Captura: Eso es lo que usa servidor.js para conectarse a MongoDB. Perfecto.



```
1 MONGO_URI=mongodb+srv://t0479022012_db_user:arely2026@cluster0.widjiyb.mongodb.net/techstore?retryWrites=true&w=majority&appName=Cluster0
```

Salida

Mostrar salida de: GitHub Copilot

GitHub Copilot 1.147.0.0 (v1.147.0.0@218a5ecac)
GitHub Copilot Agent 1.147.0 (1.147.0)
[DEBUG] [agent] [2026-07-19T02:14:00.999Z] Agent service starting
[DEBUG] [agent] [2026-07-19T02:14:01.084Z] Telemetry initialized
Auth Status: NotSignedIn

2. Identificar cuáles son los comportamientos esperados de esa parte del aplicativo.

Captura

#

3



Los comportamientos esperados del aplicativo TechStore son:

1. Conexión exitosa: Al ejecutar node servidor.js debe mostrar el mensaje  Conectado a MongoDB Local sin errores.
2. Inicio del servidor: Debe mostrar  Servidor corriendo en http://localhost:3000
3. Visualización de productos: Al acceder desde el navegador, la interfaz web debe cargar y mostrar correctamente los productos almacenados en la base de datos: nombre, descripción, precio y botón "Agregar al Carrito".
4. Interfaz funcional: El diseño debe ser responsive, con tema oscuro y el contador del carrito visible en la parte superior.
5. Sin errores: No debe presentar errores en consola del navegador ni en la terminal.

Evidencia: Como se observa en la captura, la tienda cargó correctamente los 3 productos desde MongoDB, cumpliendo con todos los comportamientos esperados

Captura #5 productos.js - Datos y lógica de TechStore

1. Simular el entorno de prueba en cuenta los datos que necesita
2. Registra los resultados de la prueba

LÓGICA Y DATOS DE PRODUCTOS - productos.js

En la imagen se observa el archivo productos.js donde se define el arreglo de productos con 6 ítems de prueba: Laptop Gamer ASUS ROG, PC Gamer Ryzen 5, Monitor 27", Teclado Mecánico, Mouse Logitech y Audífonos HyperX. También se define el arreglo carrito y la función mostrarProductos() que pinta las cards en el grid de la tienda.

3: LÓGICA Y DATOS DE PRODUCTOS - productos.js

En la imagen se observa el archivo productos.js donde se define el arreglo de productos con 6 ítems de prueba: Laptop Gamer ASUS ROG, PC Gamer Ryzen 5, Monitor 27", Teclado Mecánico, Mouse Logitech y Audífonos HyperX. También se define el arreglo carrito y la función mostrarProductos() que pinta las cards en el grid de la tienda.

Captura de pantalla de instalación de node servidor.js

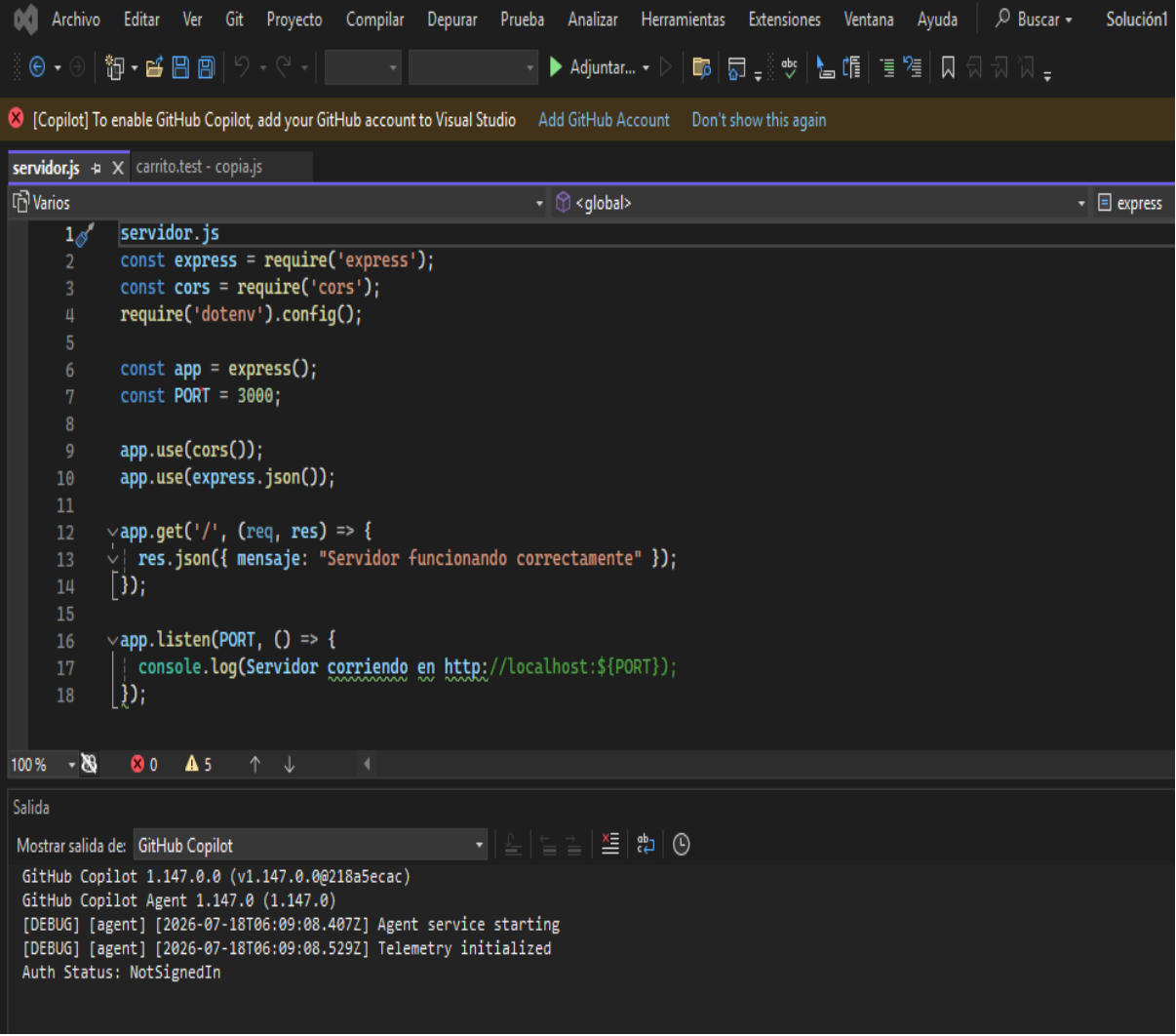
-

The image shows a Visual Studio Code editor window with a file named `servidor.js` open. The code is written in JavaScript and uses ES6 arrow functions. It includes a connection to MongoDB Atlas, a root endpoint that returns a success message, and an API endpoint for products. The server is configured to listen on port 3000.

```
13     .then(() => console.log('✅ Conectado a MongoDB Atlas'))
14     .catch(err => console.log(err));
15
16     app.get('/', (req, res) => {
17         res.send("TechStore funcionando!");
18     });
19
20     app.get('/api/productos', (req, res) => {
21         res.json([
22             { nombre: "Laptop Gamer", precio: 1200, imagen: "https://via.placeholder.com/300" },
23             { nombre: "Teclado Mecánico", precio: 80, imagen: "https://via.placeholder.com/300" },
24             { nombre: "Mouse RGB", precio: 45, imagen: "https://via.placeholder.com/300" }
25         ]);
26     });
27
28     const PORT = 3000;
29     app.listen(PORT, () => {
30         console.log(Servidor corriendo en http://localhost:${PORT});
31     });
```

Below the editor, a PowerShell terminal window is open, displaying the startup logs of Node.js v24.18.0. The prompt shows the current directory is `C:\Users\arely\OneDrive\Desktop\Logo.TechStore>`. The terminal has a title bar that says "PowerShell para desarrolladores" and a "Salida" button.

captura de levantando el servidor en visual



The image shows a screenshot of the Visual Studio Code editor interface. The top menu bar includes options like Archivo, Editar, Ver, Git, Proyecto, Compilar, Depurar, Prueba, Analizar, Herramientas, Extensiones, Ventana, Ayuda, and a search bar. Below the menu is a toolbar with various icons. A notification bar at the top states: "[Copilot] To enable GitHub Copilot, add your GitHub account to Visual Studio. Add GitHub Account Don't show this again".

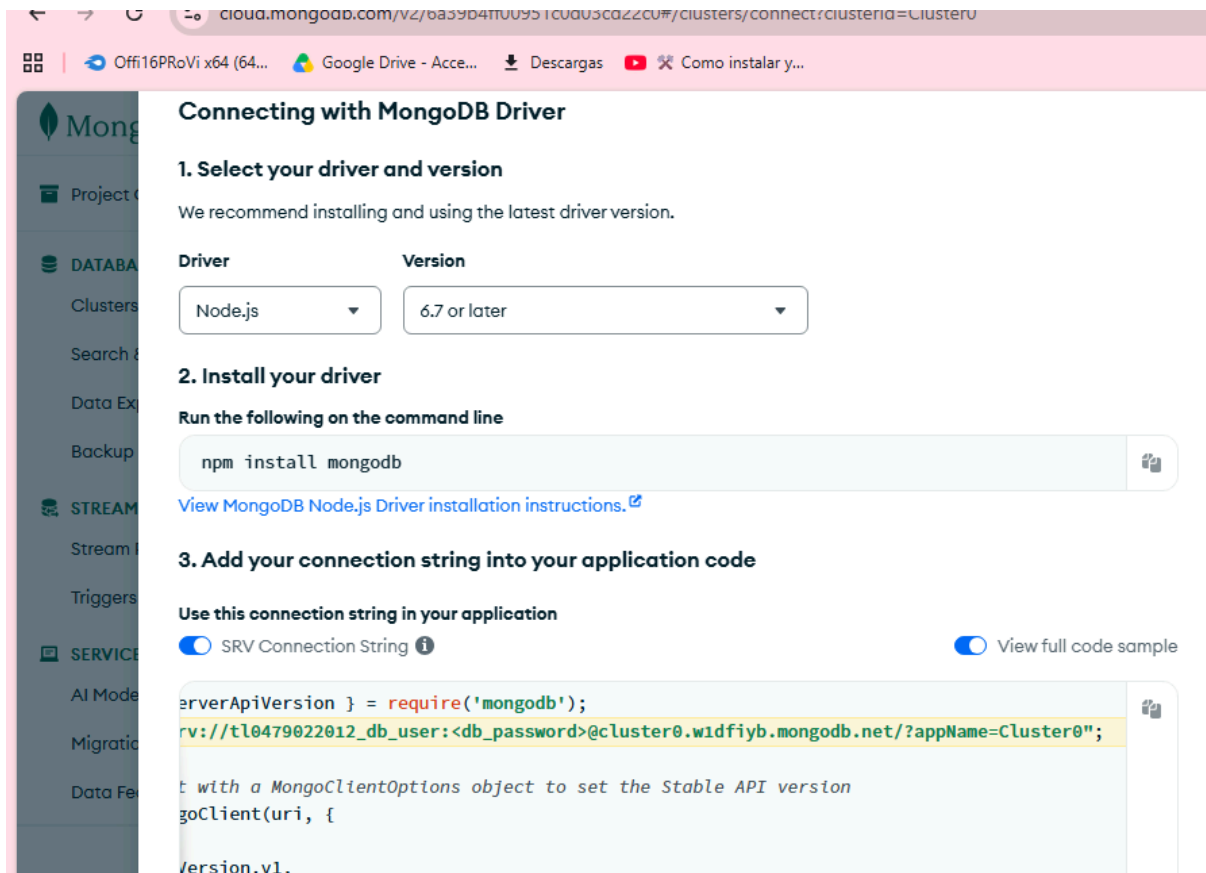
The editor window displays a file named `servidor.js` with the following code:

```
1 const express = require('express');
2 const cors = require('cors');
3 require('dotenv').config();
4
5
6 const app = express();
7 const PORT = 3000;
8
9 app.use(cors());
10 app.use(express.json());
11
12 app.get('/', (req, res) => {
13   res.json({ mensaje: "Servidor funcionando correctamente" });
14 });
15
16 app.listen(PORT, () => {
17   console.log(Servidor corriendo en http://localhost:${PORT});
18 });
```

Below the editor, the Output console is visible, showing the following text:

```
Mostrar salida de: GitHub Copilot
GitHub Copilot 1.147.0.0 (v1.147.0.0@218a5ecac)
GitHub Copilot Agent 1.147.0 (1.147.0)
[DEBUG] [agent] [2026-07-18T06:09:08.407Z] Agent service starting
[DEBUG] [agent] [2026-07-18T06:09:08.529Z] Telemetry initialized
Auth Status: NotSignedIn
```

Captura a conexión a mongodb Atlas



cloud.mongodb.com/v2/ba39b4100951c0d03ca22cu#/clusters/connect?clusterId=Cluster0

Offi16ProVi x64 (64... Google Drive - Acce... Descargas Como instalar y...

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver	Version
Node.js	6.7 or later

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String [View full code sample](#)

```
serverApiVersion } = require('mongodb');  
rv://tl0479022012_db_user:<db_password>@cluster0.widfiyb.mongodb.net/?appName=Cluster0";  
  
t with a MongoClientOptions object to set the Stable API version  
goClient(uri, {  
Version.v1.
```

[node.js](#) en mongo Atlas

Captura de pantalla

Connect to Cluster0



Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver	Version
Node.js	6.7 or later

2. Install your driver

Run the following on the command line

```
npm install mongodb
```



[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String ⓘ

☐ View full code sample

```
const { MongoClient, ServerApiVersion } = require('mongodb');
const uri = "mongodb+srv://tl0479022012_db_user:<db_password>@cluster0.w1dfiyb.mongodb.net";

// Create a MongoClient with a MongoClientOptions object to set the Stable API version
const client = new MongoClient(uri, {
  serverApi: {
    version: ServerApiVersion.v1,
    strict: true,
    deprecationErrors: true,
  }
});

async function run() {
  try {
    // Connect the client to the server (optional starting in v4.7)
```

```

try {
  // Connect the client to the server (optional starting in v4.7)
  await client.connect();
  // Send a ping to confirm a successful connection
  await client.db("admin").command({ ping: 1 });
  console.log("Pinged your deployment. You successfully connected to MongoDB!");
} finally {
  // Ensures that the client will close when you finish/error
  await client.close();
}
}
run().catch(console.dir);

```

Replace `<db_password>` with the password for the `tl0479022012_db_user` database user. Ensure any option params are [URL encoded](#).

RESOURCES

[Get started with the Node.js Driver](#)

[Node.js Starter Sample App](#)

[Access your Database Users](#)

[Troubleshoot Connections](#)

Go Back

Done

Captura de conectando el cluster en mongo ATLAS

Project Overview

DATABASE

Clusters

Search & Vector Search

Data Explorer

Backup

STREAMING DATA

Stream Processing

Triggers

SERVICES

AI Models

Migration

Data Federation

ORGANIZATION

Dina Arely's Org - 20...

PROJECT

Project 0

?

DA

ⓘ

Your organization does not have designated security and operations contacts. Add contact information in [Organization Settings](#) to receive notifications.

✕

Project 0 Overview

Clusters

Create cluster

...

Cluster0

Connect

Edit configuration

Monitoring for Cluster0 is paused.

Monitoring will automatically resume when you connect to your cluster.

[Visit the documentation for more info](#)

23

CONCLUSIÓN

En esta primera entrega del proyecto TechStore se logró diseñar el esquema de base de datos en MongoDB y definir la integración con el backend en Node.js y Express. Se implementó la lógica de cálculo del carrito y se validó mediante pruebas unitarias con Jest. Además, se diseñaron los prototipos de las interfaces web principales para garantizar una buena experiencia de usuario. Estos avances sientan las bases para el desarrollo de los módulos de autenticación, pago y administración en las siguientes entregas.

Pruebas Unitarias

1. Aumentar cobertura*: Se recomienda alcanzar al menos 80% de cobertura de código.
Actualmente solo se probaron las rutas GET, falta probar POST, PUT y DELETE.
2. Automatizar pruebas*: Implementar un script en package.json como
`"test": "jest"` para ejecutar todas las pruebas con un solo comando.
3. Base de datos de prueba*: Usar una base de datos de MongoDB separada solo para pruebas, para no afectar los datos reales de producción.
4. Probar casos de error*: Además de probar cuando todo funciona bien, se deben probar respuestas cuando el servidor falla, cuando no hay conexión a BD o cuando se envían datos inválidos.

RECOMENDACIONES FINALES

Pruebas Unitarias

1. ***Aumentar cobertura*:** Se recomienda alcanzar al menos 80% de cobertura de código.
Actualmente solo se probaron las rutas GET, falta probar POST, PUT y DELETE.
2. **Automatizar pruebas*:** Implementar un script en package.json como
"test": "jest" para ejecutar todas las pruebas con un solo comando.
3. **Base de datos de prueba*:** Usar una base de datos de MongoDB separada solo para pruebas, para no afectar los datos reales de producción.
4. **Probar casos de error*:** Además de probar cuando todo funciona bien, se deben probar respuestas cuando el servidor falla, cuando no hay conexión a BD o cuando se envían datos inválidos.